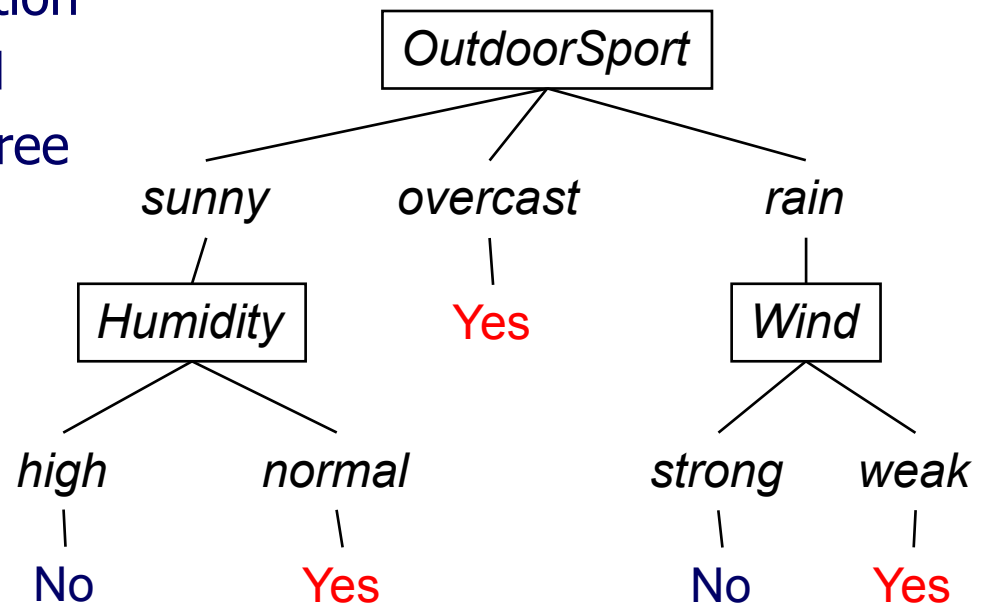


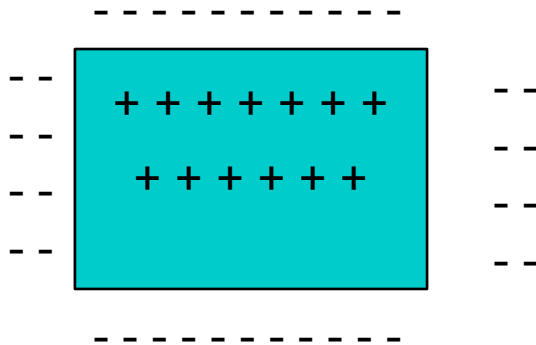
Decision Trees

Representation of Concepts

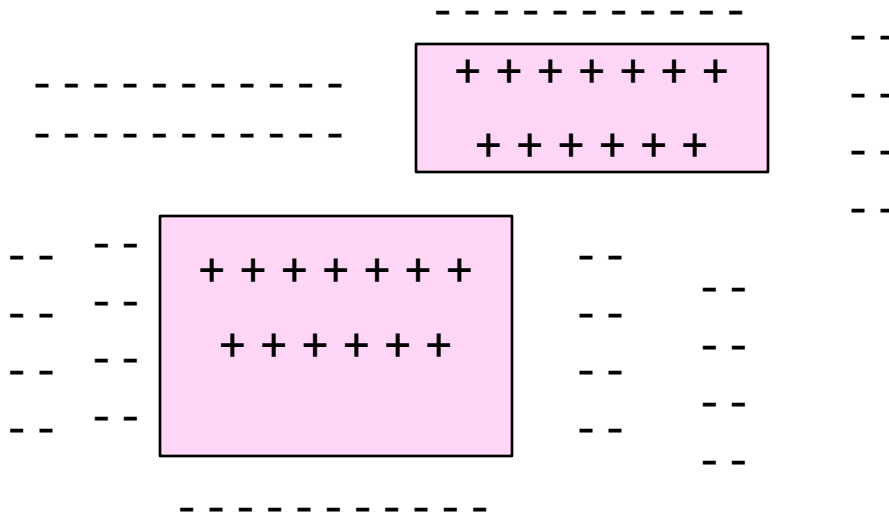
- < Concept learning: conjunction of attributes
 - < (Sunny AND Hot AND Humid AND Windy) +
- Decision trees: disjunction of conjunction of attributes
 - (Sunny AND Normal) OR (Overcast) OR (Rain AND Weak) +
 - More powerful representation
 - Larger hypothesis space H
 - Can be represented as a tree
 - Common form of decision making in humans



Rectangle learning....



Conjunctions
(single rectangle)



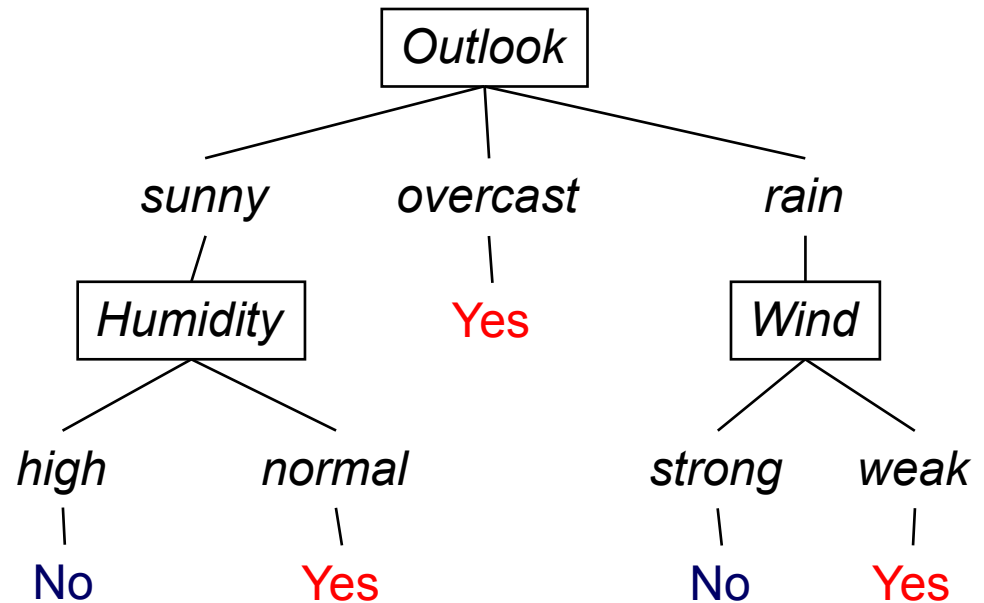
Disjunctions of Conjunctions
(union of rectangles)

Training Examples

Day	Outlook	Temp	Humidity	Wind	Tennis?
<i>D1</i>	Sunny	Hot	High	Weak	<i>No</i>
<i>D2</i>	Sunny	Hot	High	Strong	<i>No</i>
<i>D3</i>	Overcast	Hot	High	Weak	<i>Yes</i>
<i>D4</i>	Rain	Mild	High	Weak	<i>Yes</i>
<i>D5</i>	Rain	Cool	Normal	Weak	<i>Yes</i>
<i>D6</i>	Rain	Cool	Normal	Strong	<i>No</i>
<i>D7</i>	Overcast	Cool	Normal	Strong	<i>Yes</i>
<i>D8</i>	Sunny	Mild	High	Weak	<i>No</i>
<i>D9</i>	Sunny	Cool	Normal	Weak	<i>Yes</i>
<i>D10</i>	Rain	Mild	Normal	Weak	<i>Yes</i>
<i>D11</i>	Sunny	Mild	Normal	Strong	<i>Yes</i>
<i>D12</i>	Overcast	Mild	High	Strong	<i>Yes</i>
<i>D13</i>	Overcast	Hot	Normal	Weak	<i>Yes</i>
<i>D14</i>	Rain	Mild	High	Strong	<i>No</i>

Decision Trees

- Decision tree to represent learned target functions
 - Each internal node tests an attribute
 - Each branch corresponds to attribute value
 - Each leaf node assigns a classification
- Can be represented by logical formulas



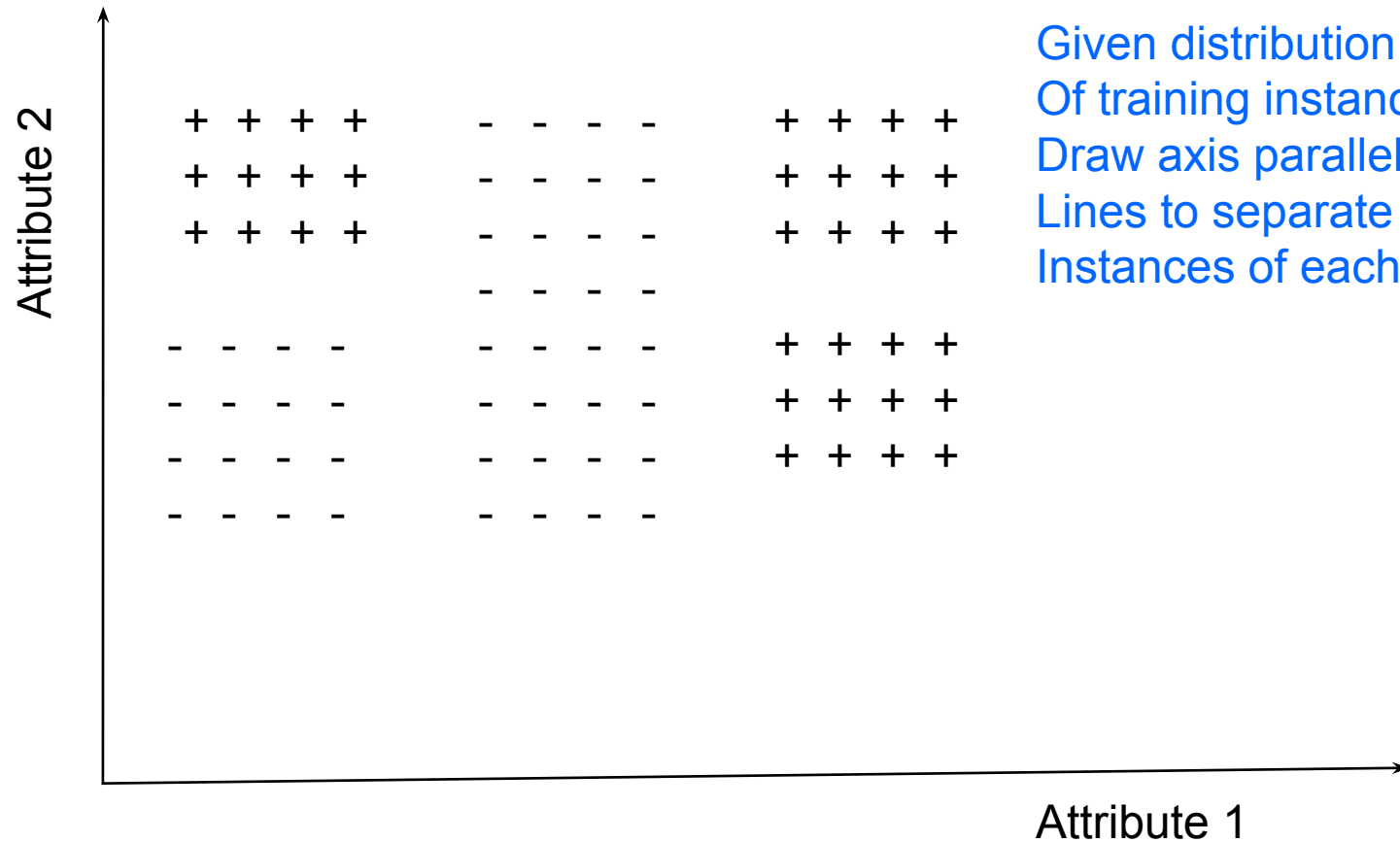
Representation in decision trees

- < Example of representing rule in DT's:
if outlook = sunny AND humidity = normal
OR
if outlook = overcast
OR
if outlook = rain AND wind = weak
then playtennis

Applications of Decision Trees

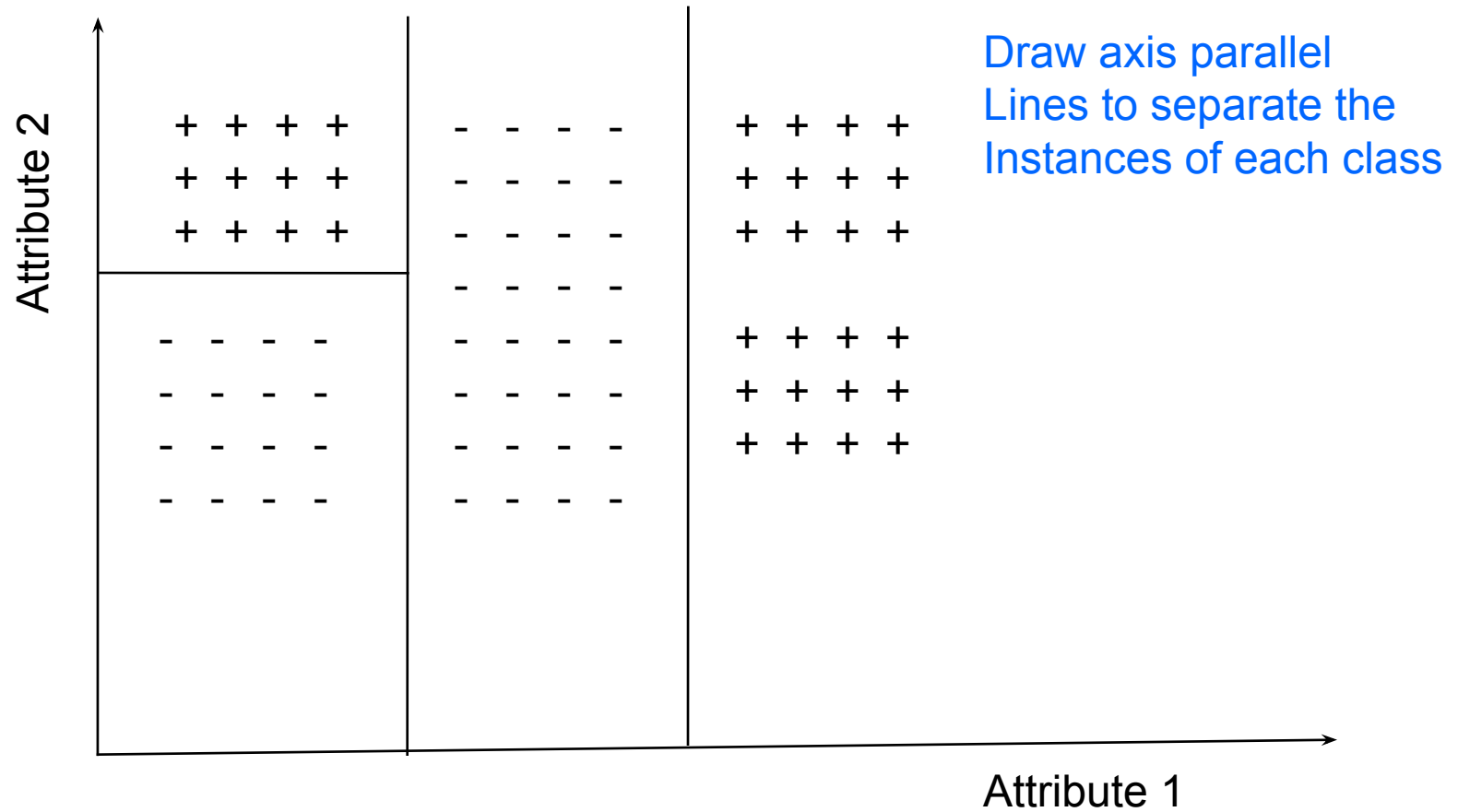
- < Instances describable by a fixed set of attributes and their values
- < Target function is discrete valued
 - 2-valued
 - N-valued
 - But can approximate continuous functions
- < Disjunctive hypothesis space
- < Possibly noisy training data
 - Errors, missing values, ...
- < Examples:
 - Equipment or medical diagnosis
 - Credit risk analysis
 - Calendar scheduling preferences

Decision Trees

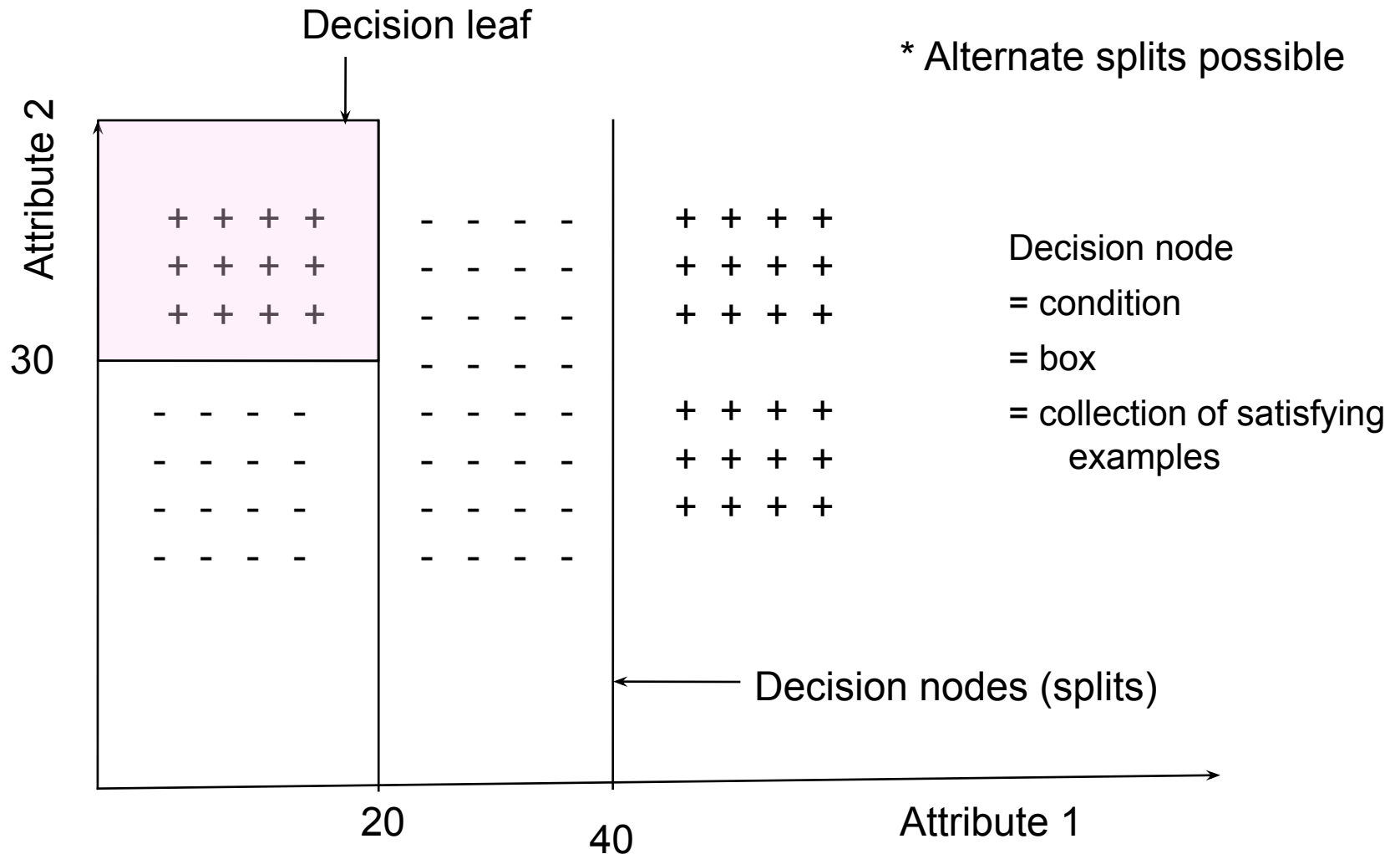


Given distribution
Of training instances
Draw axis parallel
Lines to separate the
Instances of each class

Decision Tree Structure



Decision Tree Structure



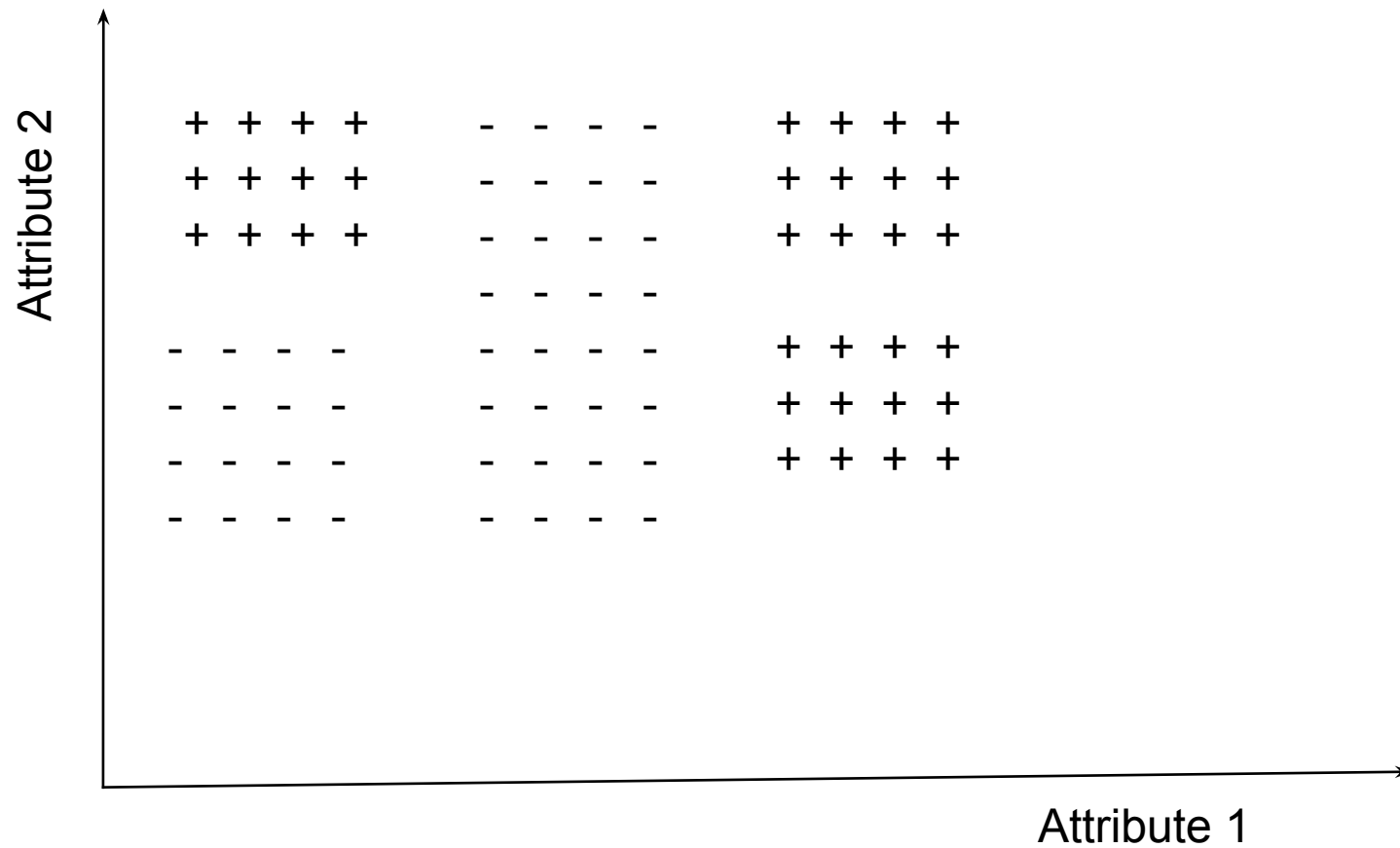
Decision Tree Construction

- Find the best structure
- Given a training data set

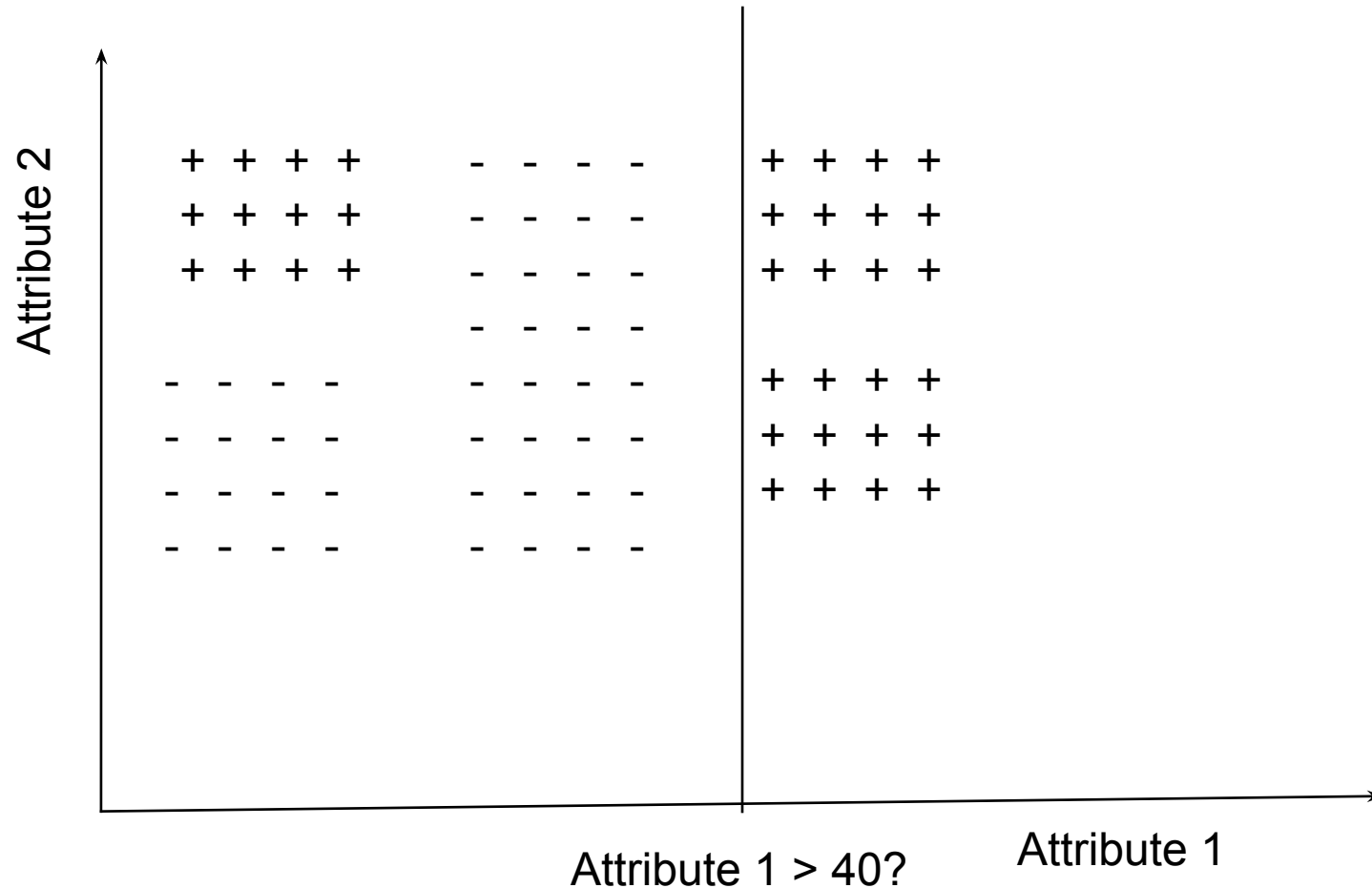
Top-Down Construction

- < Start with empty tree
- < Main loop:
 1. Split the “best” decision attribute (A) for next node
 2. Assign A as decision attribute for node
 3. For each value of A , create new descendant of node
 4. Sort training examples to leaf nodes
 5. If training examples perfectly classified, STOP,
 Else iterate over new leaf nodes
- < Grow tree just deep enough for perfect classification
 - If possible (or can approximate at chosen depth)
- < Which attribute is best?

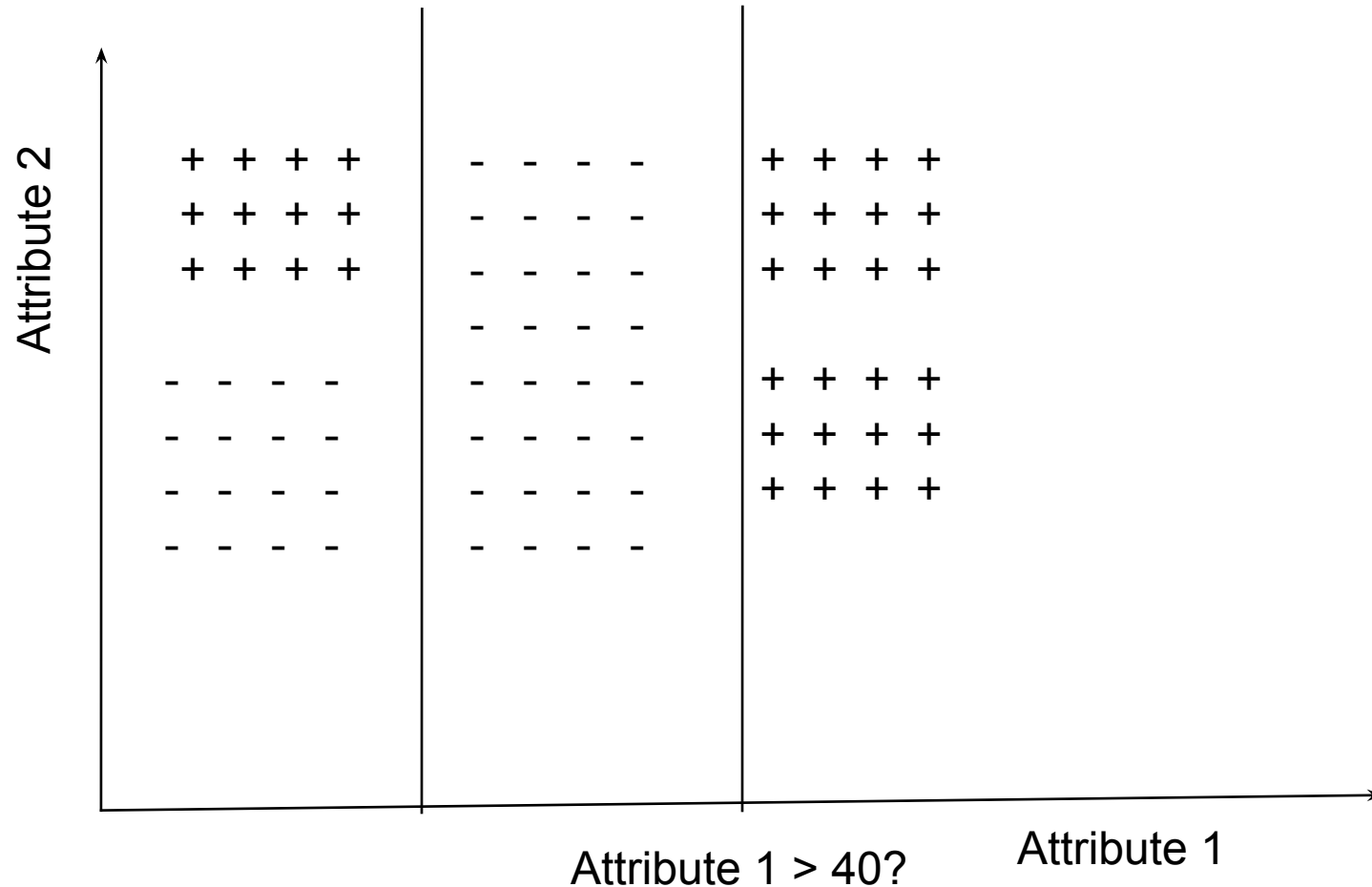
Best attribute to split?



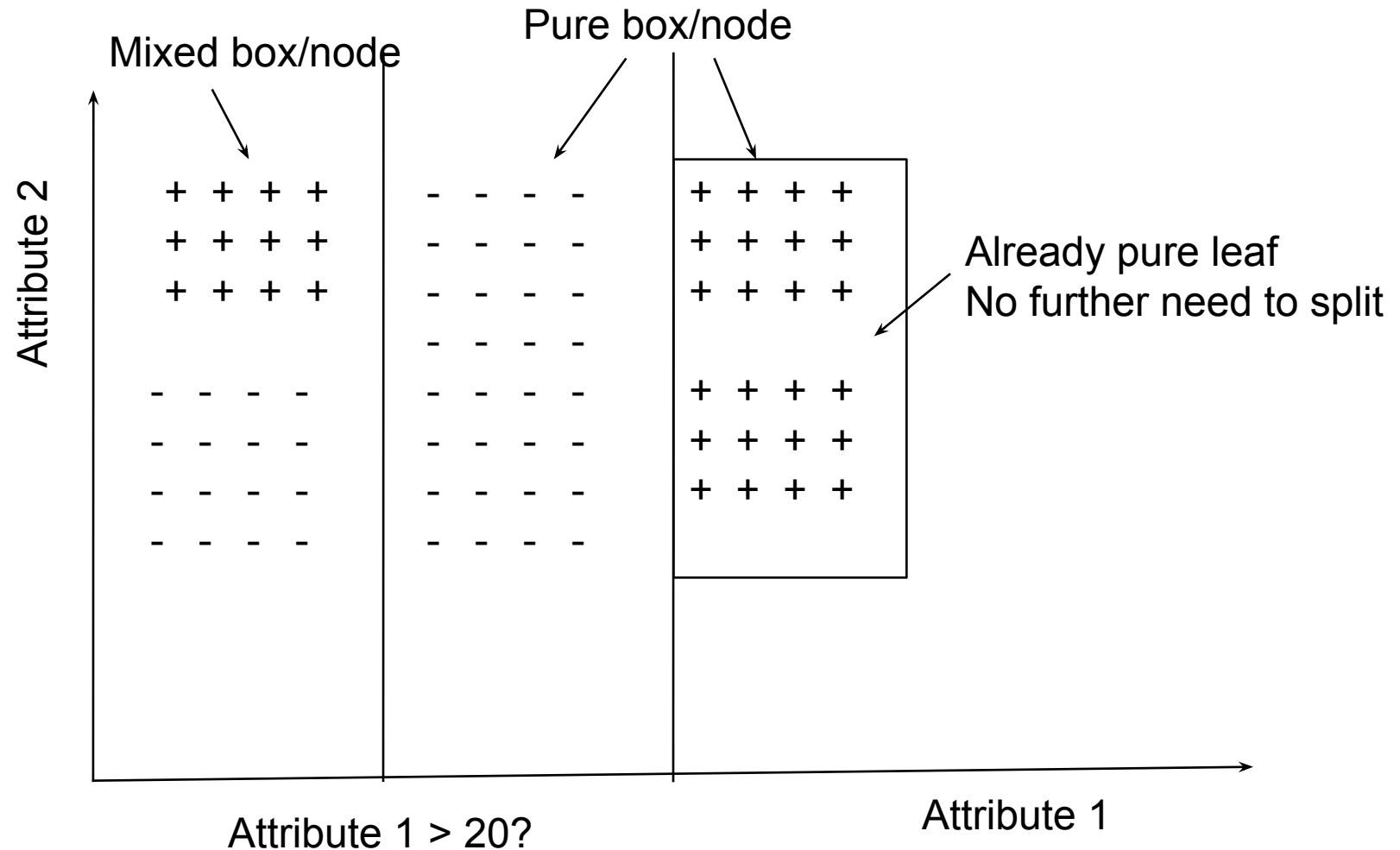
Best attribute to split?



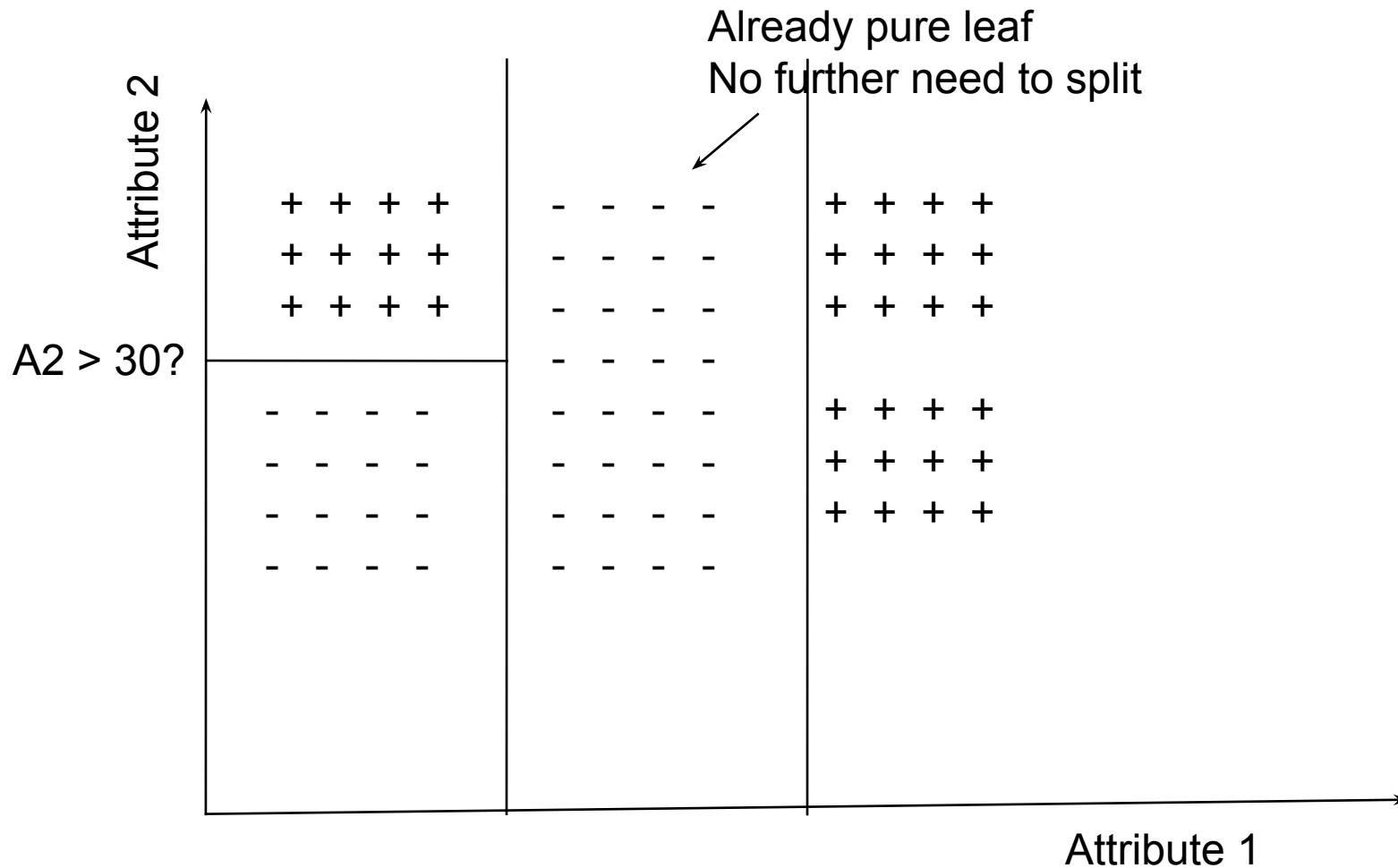
Best attribute to split?



Which split to make next?



Which split to make next?

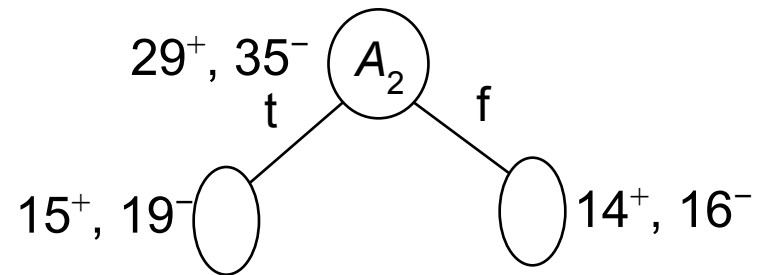
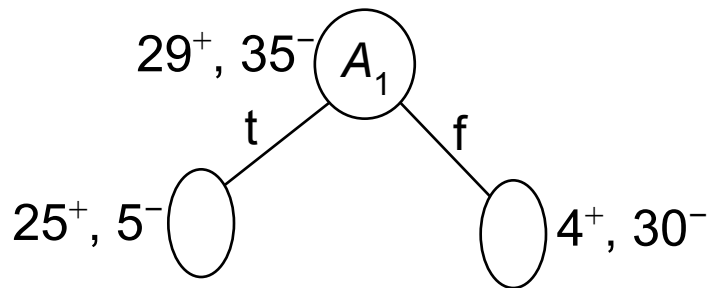


Principle of Decision Tree Construction

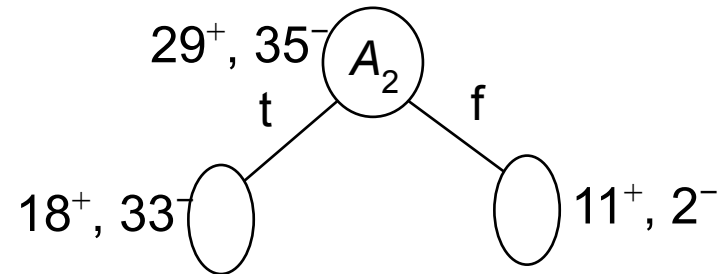
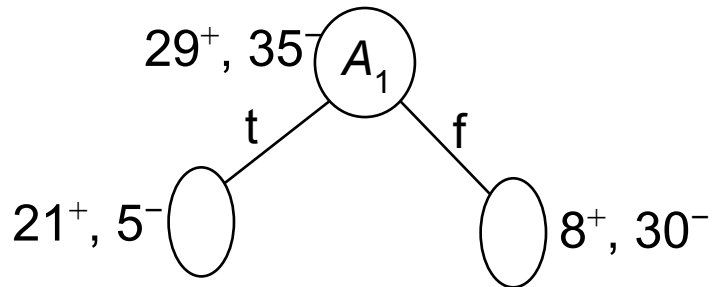
- Finally we want to form pure leaves
 - Correct classification
- Greedy approach to reach correct classification
 1. Initially treat the entire data set as a single box
 2. For each box choose the split that reduces its impurity (in terms of class labels) by the maximum amount
 3. Split the box having highest reduction in impurity
 4. Continue to Step 2
 5. Stop when all boxes are pure

Choosing Best Attribute?

- Consider 64 examples, 29^+ and 35^-
- Which one is better?



- Which is better?



Entropy

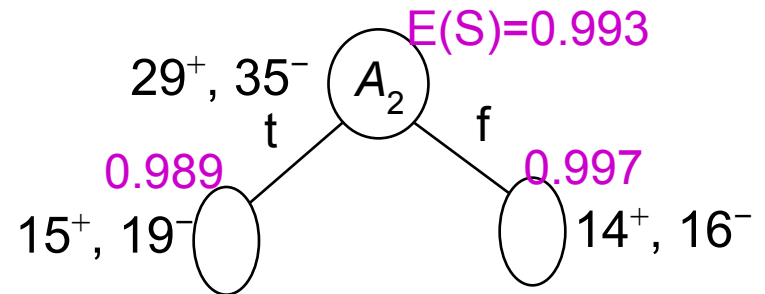
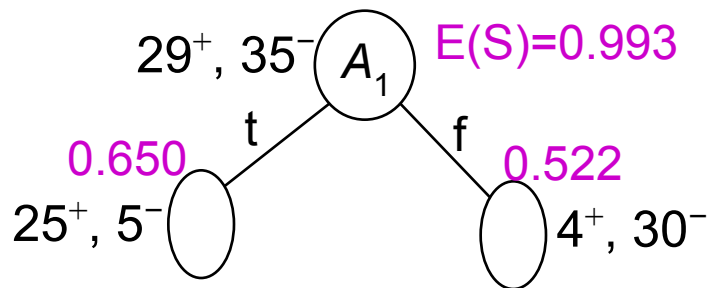
- A measure for
 - uncertainty
 - purity
 - information content
- Information theory: optimal length code assigns $(-\log_2 p)$ bits to message having probability p
- S is a sample of training examples
 - p_+ is the proportion of positive examples in S
 - p_- is the proportion of negative examples in S
- Entropy of S : average optimal number of bits to encode information about certainty/uncertainty about S
$$\text{Entropy}(S) = p_+(-\log_2 p_+) + p_-(-\log_2 p_-) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$
- Can be generalized to more than two values

Entropy

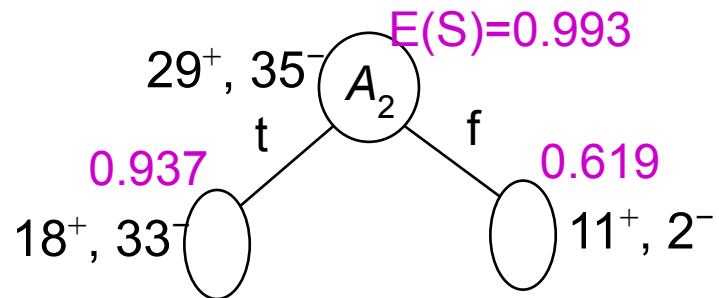
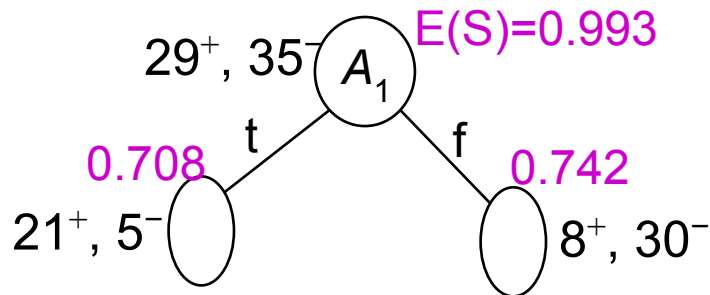
- < Entropy can also be viewed as measuring
 - purity of S ,
 - uncertainty in S ,
 - information in S , ...
- < E.g.: values of entropy for $p_+=1$, $p_+=0$, $p_+=.5$
- < Easy generalization to more than binary values
 - Sum over $p_i * (-\log_2 p_i)$, $i=1,n$
 - ❖ i is + or – for binary
 - ❖ i varies from 1 to n in the general case

Choosing Best Attribute?

- Consider 64 examples ($29^+, 35^-$) and compute entropies:
- Which one is better?



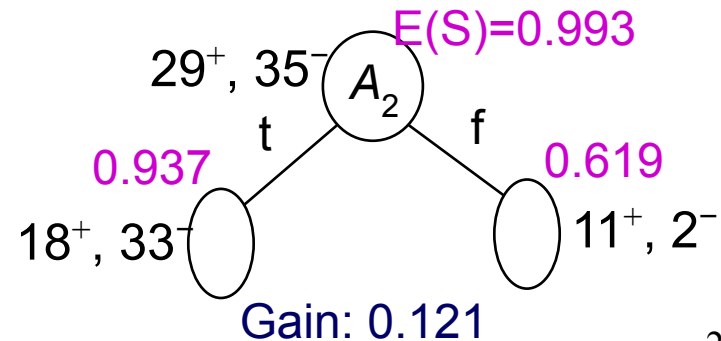
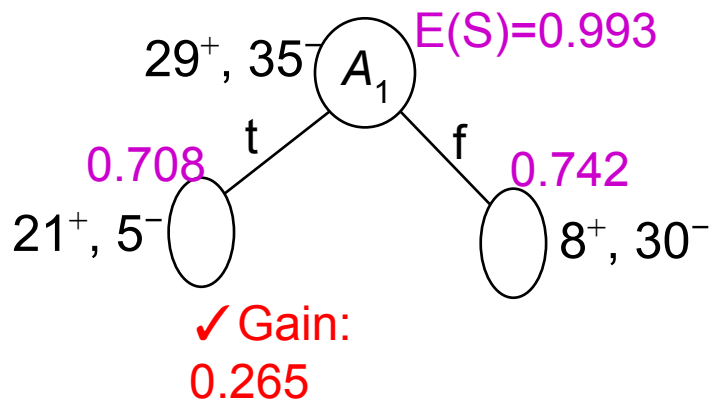
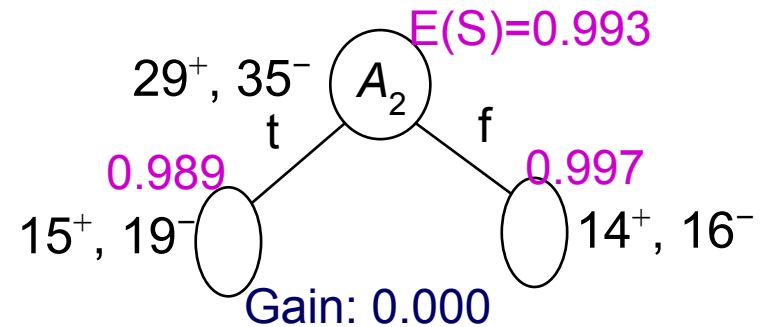
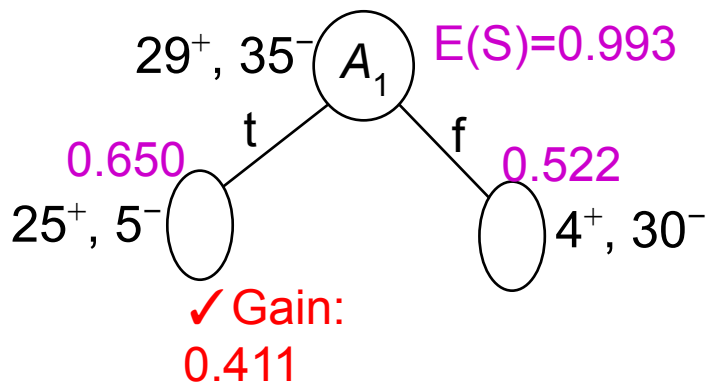
- Which is better?



Information Gain

- $Gain(S, A)$: reduction in entropy after choosing attr. A

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$



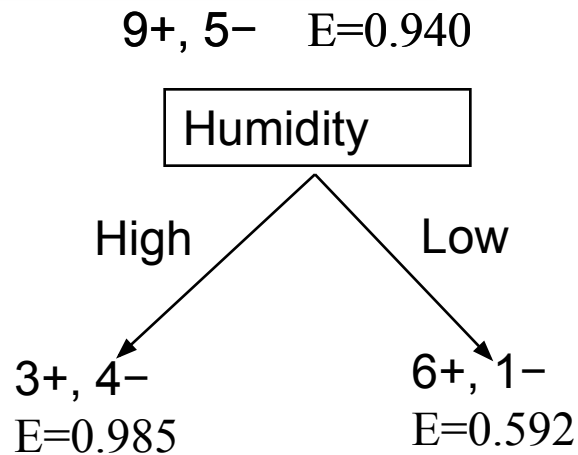
Gain function

- < Gain is measure of how much can
 - Reduce uncertainty
 - ❖ Value lies between 0,1
 - ❖ What is significance of
 - gain of 0?
 - example where have 50/50 split of +/- both before *and* after discriminating on attributes values
 - gain of 1?
 - Example of going from “perfect uncertainty” to perfect certainty after splitting example with predictive attribute
 - Find “patterns” in TE’s relating to attribute values
 - ❖ Move to locally minimal representation of TE’s

Training Examples

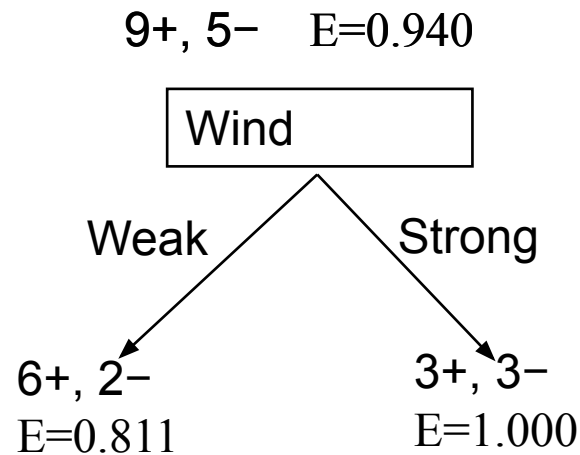
Day	Outlook	Temp	Humidity	Wind	Tennis?
<i>D1</i>	Sunny	Hot	High	Weak	<i>No</i>
<i>D2</i>	Sunny	Hot	High	Strong	<i>No</i>
<i>D3</i>	Overcast	Hot	High	Weak	<i>Yes</i>
<i>D4</i>	Rain	Mild	High	Weak	<i>Yes</i>
<i>D5</i>	Rain	Cool	Normal	Weak	<i>Yes</i>
<i>D6</i>	Rain	Cool	Normal	Strong	<i>No</i>
<i>D7</i>	Overcast	Cool	Normal	Strong	<i>Yes</i>
<i>D8</i>	Sunny	Mild	High	Weak	<i>No</i>
<i>D9</i>	Sunny	Cool	Normal	Weak	<i>Yes</i>
<i>D10</i>	Rain	Mild	Normal	Weak	<i>Yes</i>
<i>D11</i>	Sunny	Mild	Normal	Strong	<i>Yes</i>
<i>D12</i>	Overcast	Mild	High	Strong	<i>Yes</i>
<i>D13</i>	Overcast	Hot	Normal	Weak	<i>Yes</i>
<i>D14</i>	Rain	Mild	High	Strong	<i>No</i>

Determine the Root Attribute



Gain (S, Humidity) = 0.151

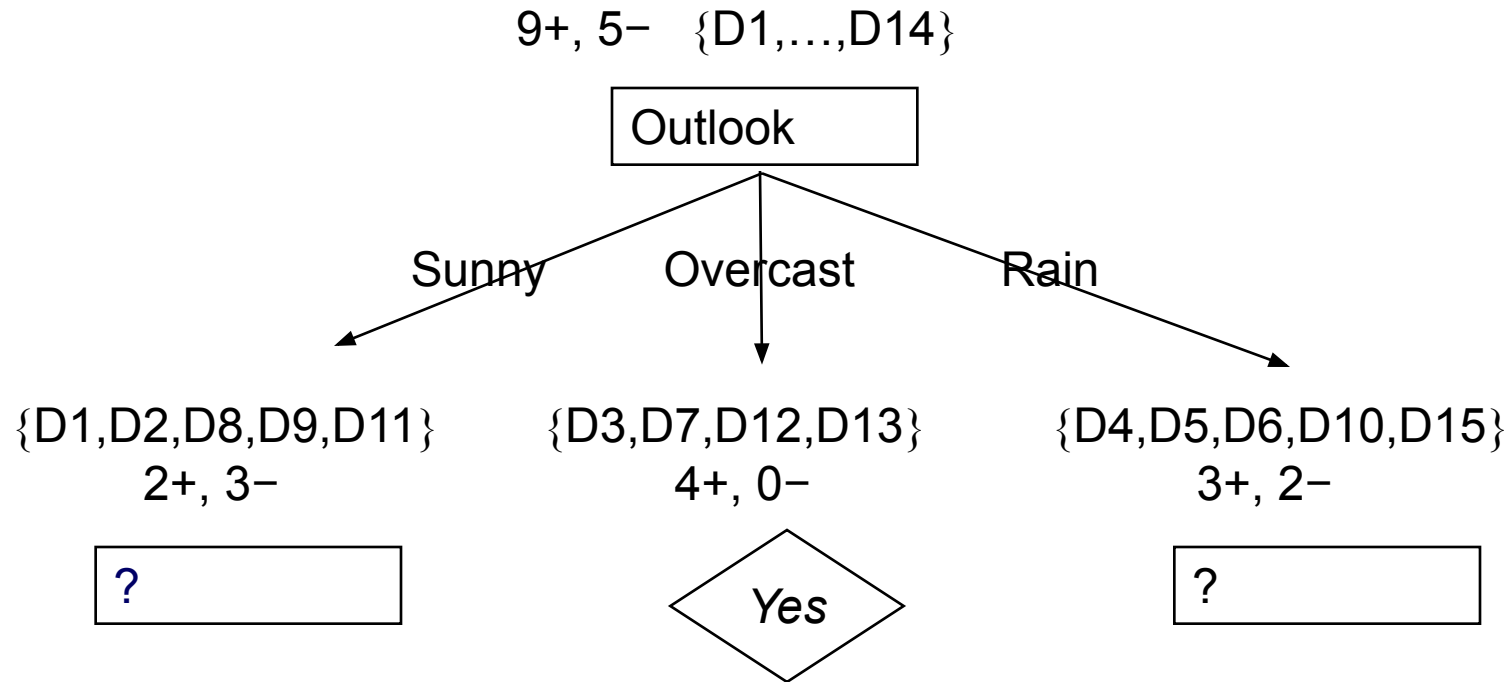
Gain (S, Outlook) = 0.246



Gain (S, Wind) = 0.048

Gain (S, Temp) = 0.029

Sort the Training Examples



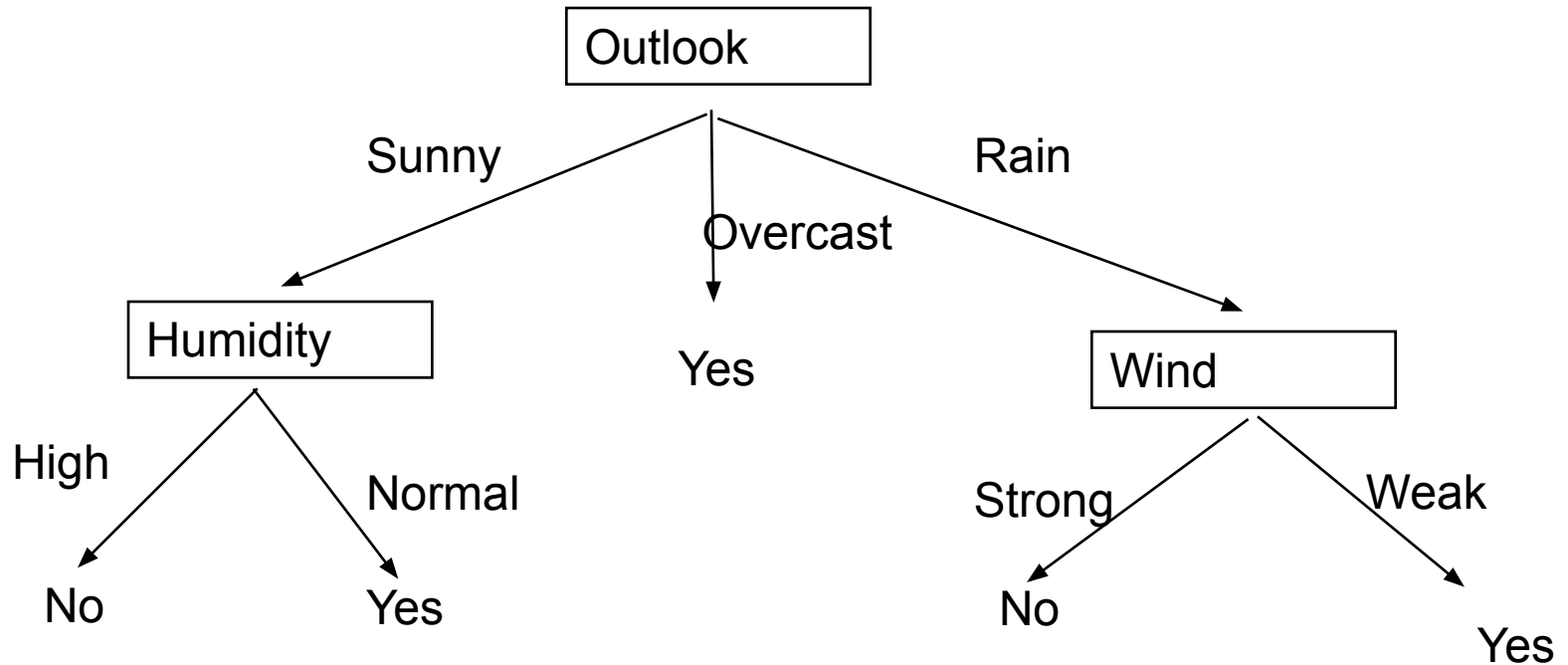
$$S_{\text{sunny}} = \{D1, D2, D8, D9, D11\}$$

$$\text{Gain}(S_{\text{sunny}}, \text{Humidity}) = .970$$

$$\text{Gain}(S_{\text{sunny}}, \text{Temp}) = .570$$

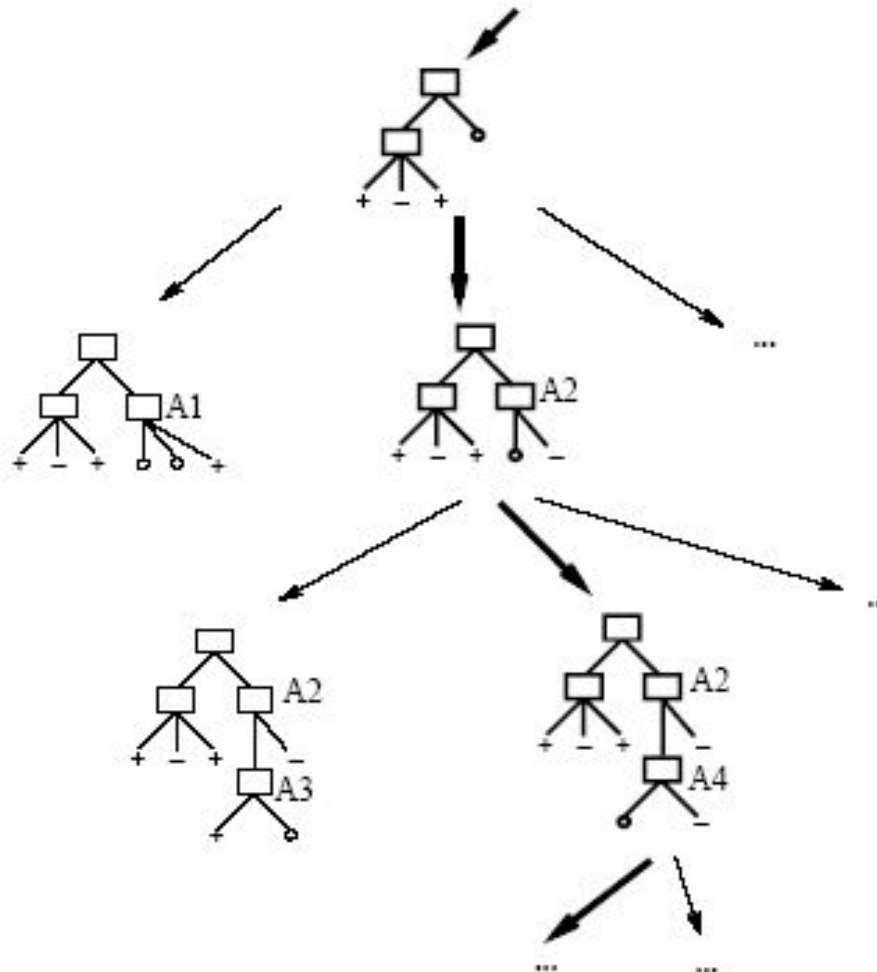
$$\text{Gain}(S_{\text{sunny}}, \text{Wind}) = .019$$

Final Decision Tree for Example



Hypothesis Space Search (ID3)

- Hypothesis space (all possible trees) is complete!
 - Target function is included in there



Hypothesis Space Search in Decision Trees

- Conduct a search of the space of decision trees which can represent all possible discrete functions.
- Goal: to find the **best** decision tree
- Finding a minimal decision tree consistent with a set of data is **NP-hard**.
- Perform a greedy heuristic search: hill climbing **without backtracking**
- Statistics-based decisions using **all data**

Hypothesis Space Search by ID3

- Hypothesis space is complete!
 - H is space of all finite DT's (all discrete functions)
 - Target function is included in there
- Simple to complex hill-climbing search of H
 - Use of gain as hill-climbing function
- Outputs a single hypothesis (which one?)
 - Cannot assess all hypotheses consistent with D (usually many)
 - Analogy to breadth first search
 - ♦ Examines all trees of given depth and chooses best...
- No backtracking
 - Locally optimal ...
- Statistics-based search choices
 - Use all TE's at each step
 - Robust to noisy data

Restriction bias vs. Preference bias

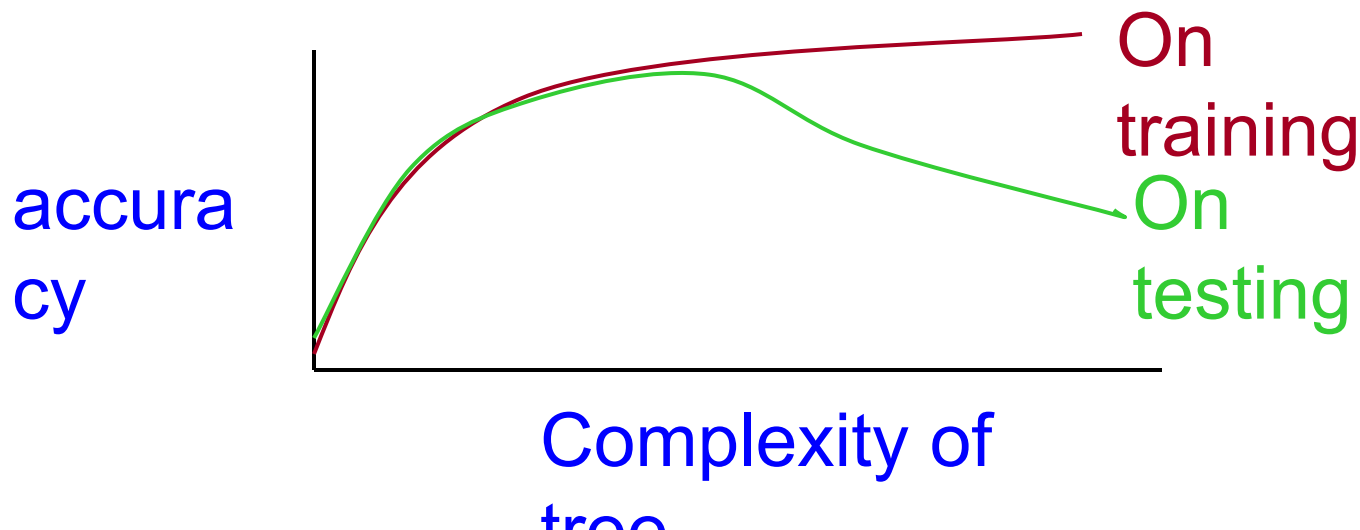
- Restriction bias (or Language bias)
 - Incomplete hypothesis space
- Preference (or search) bias
 - Incomplete search strategy
- Candidate Elimination has restriction bias
- ID3 has preference bias
- In most cases, we have both a restriction and a preference bias.

Inductive Bias in ID3

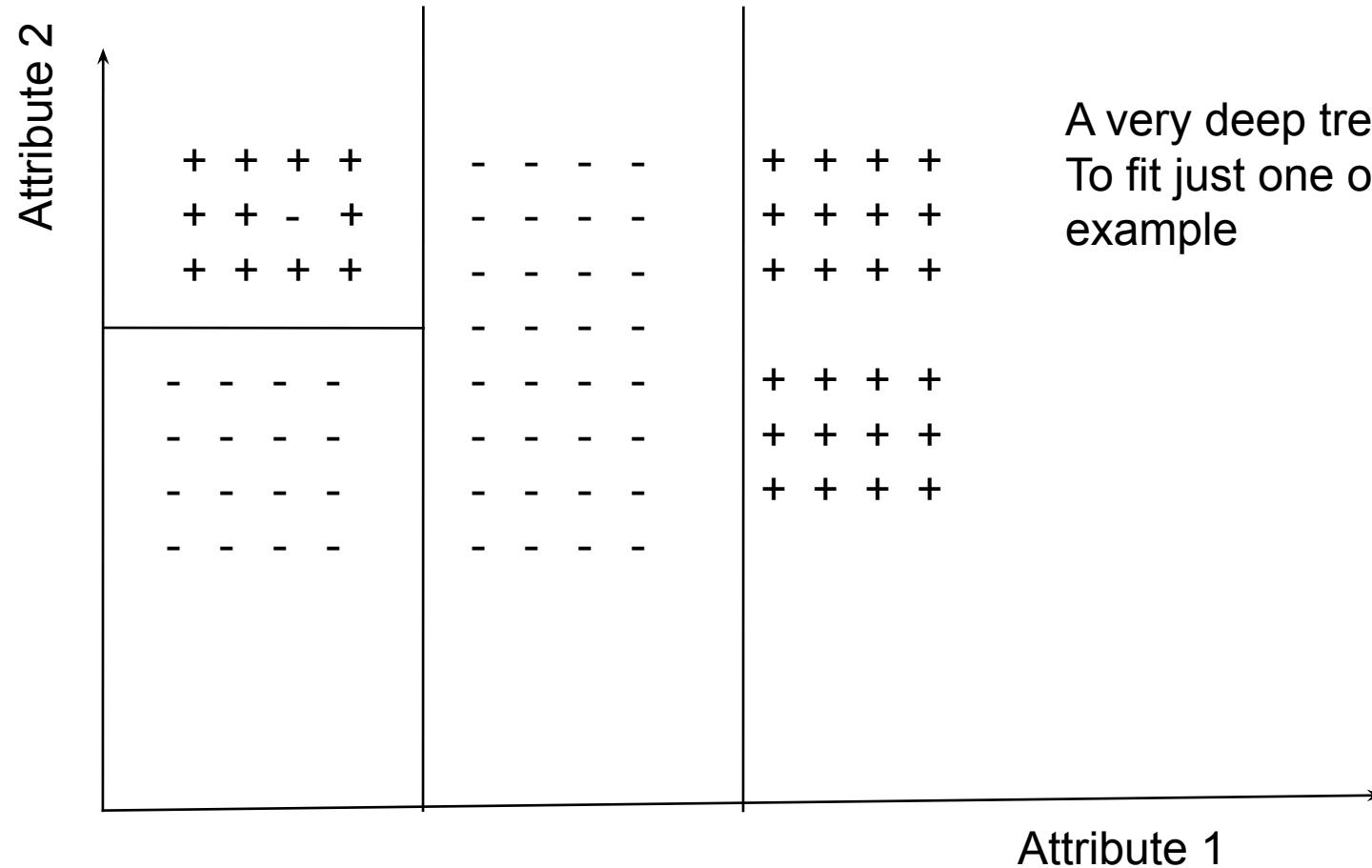
- Preference for short trees, and for those with high information gain attributes near the root
- Principle of Occam's razor
 - prefer the shortest hypothesis that fits the data
- Justification
 - Smaller likelihood of a short hypothesis fitting the data at random
- Problems
 - Other ways to reduce random fits to data
 - Size of hypothesis based on the data representation
 - ♦ Minimum description length principle

Overfitting the Data

- Learning a tree that classifies the training data perfectly may not lead to the tree with the **best generalization performance**.
 - There may be noise in the training data the tree is fitting
 - The algorithm might be making decisions based on very little data
- A hypothesis h is said to **overfit the training data** if there is another hypothesis, h' , such that h has smaller error than h' on the training data but h has larger error on the test data than h' .

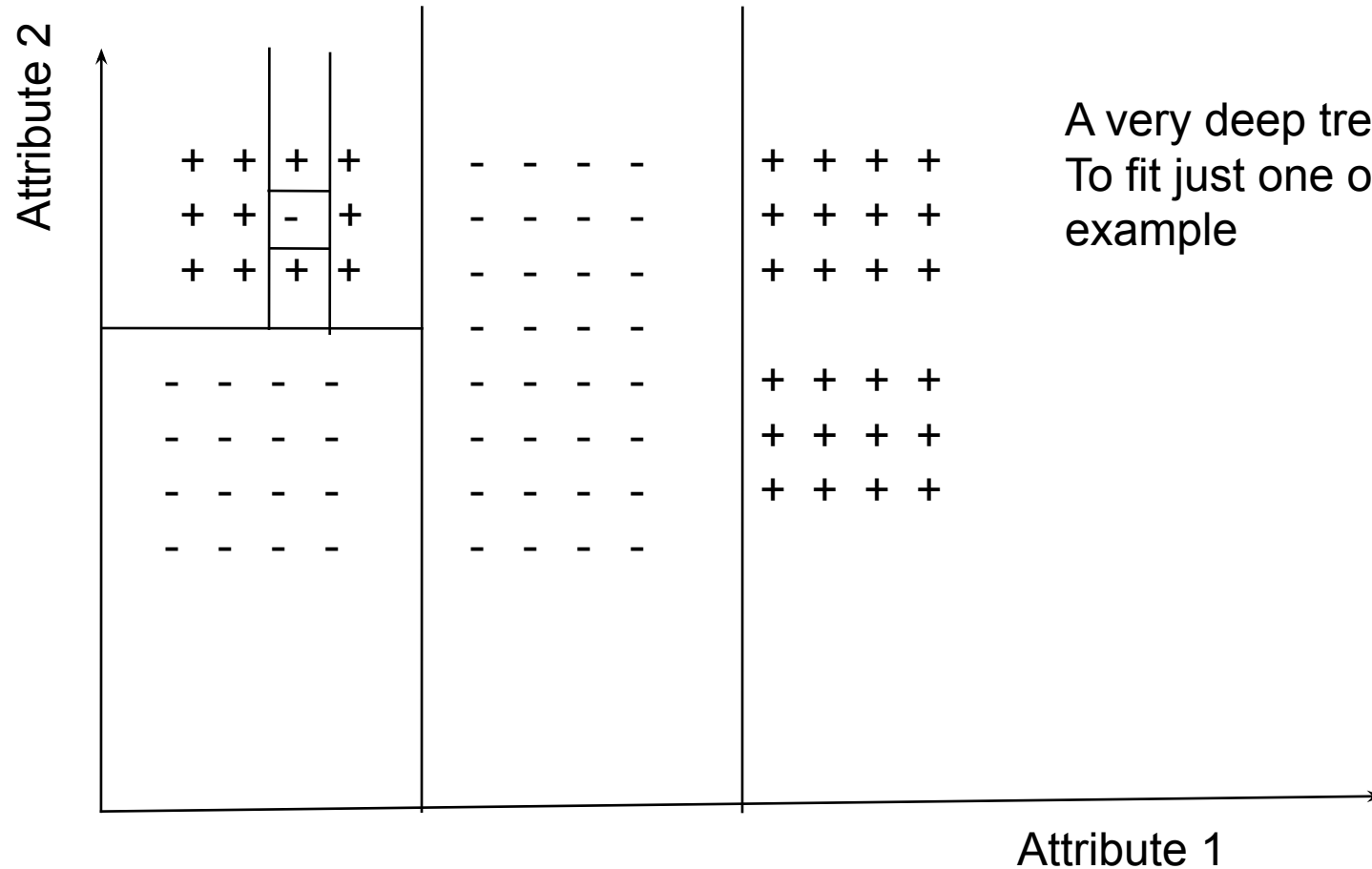


Overfitting



A very deep tree required
To fit just one odd training
example

When to stop splitting further?

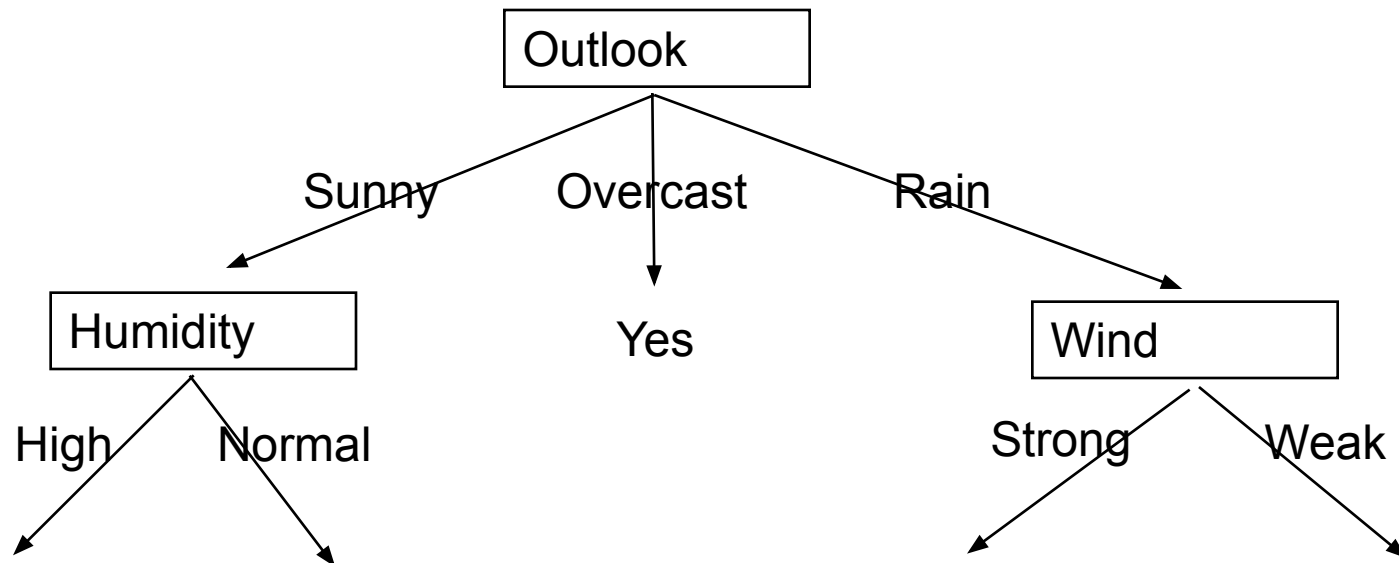


Overfitting in Decision Trees

- Consider adding *noisy* training example (should be +):

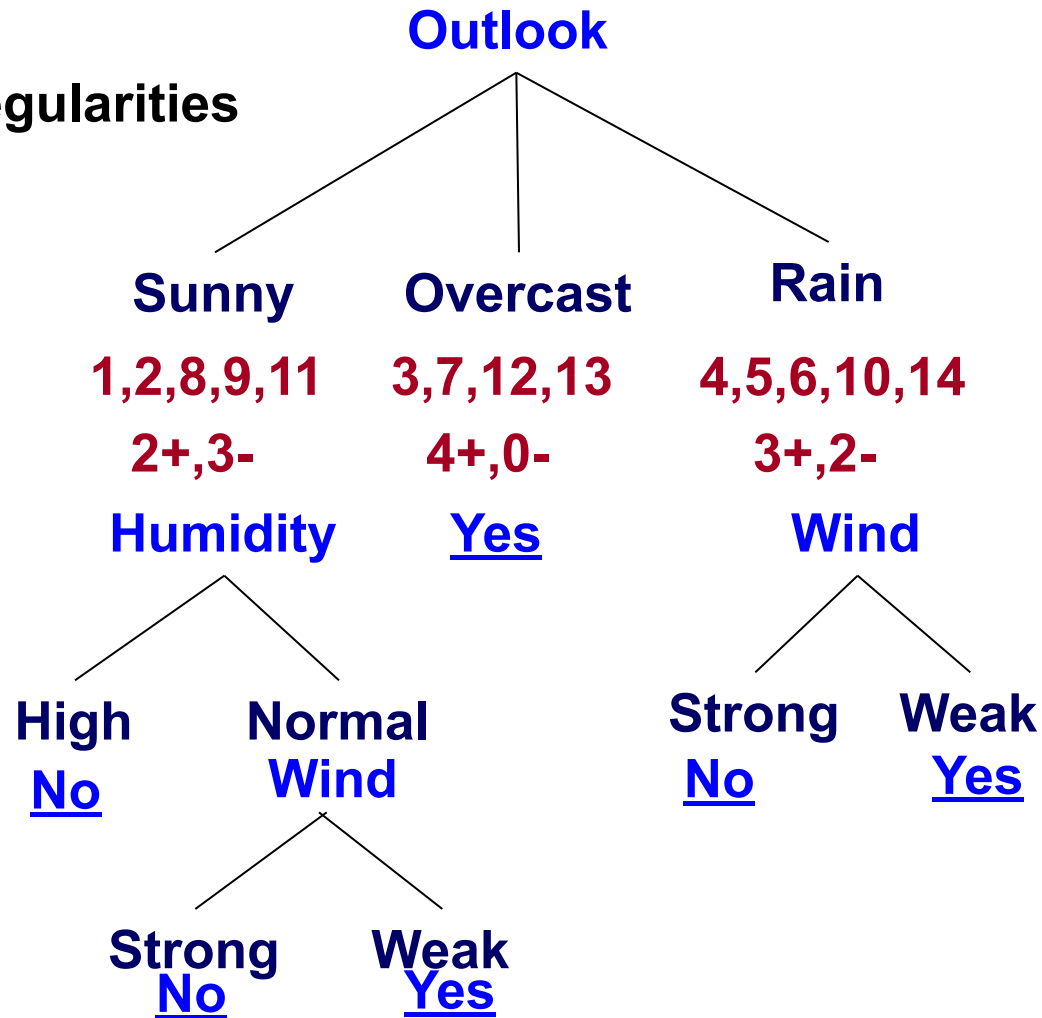
Day	Outlook	Temp	Humidity	Wind	Tennis?
<i>D15</i>	<i>Sunny</i>	<i>Hot</i>	<i>Normal</i>	<i>Strong</i>	<i>No</i>

- What effect on earlier tree?



Overfitting - Example

Noise or other
coincidental regularities



Avoiding Overfitting

- Two basic approaches
 - **Prepruning**: Stop growing the tree at some point during construction when it is determined that there is not enough data to make reliable choices.
 - **Postpruning**: Grow the full tree and then remove nodes that seem not to have sufficient evidence. (more popular)
 - Methods for evaluating subtrees to prune:
 - **Cross-validation**: Reserve hold-out set to evaluate utility (more popular)
 - **Statistical testing**: Test if the observed regularity can be dismissed as likely to be occur by chance
 - **Minimum Description Length**: Is the additional complexity of the hypothesis smaller than remembering the exceptions ?
- This is related to the notion of **regularization** that we will see in other contexts— keep the hypothesis simple.

Reduced-Error Pruning

- A post-pruning, cross validation approach
 - Partition training data into “grow” set and “validation” set.
 - Build a complete tree for the “grow” data
 - Until accuracy on validation set decreases, do:
 - For each non-leaf node in the tree
 - Temporarily prune the tree below; replace it by majority vote.
 - Test the accuracy of the hypothesis on the validation set
 - Permanently prune the node with the greatest increase in accuracy on the validation test.
- Problem: Uses less data to construct the tree
- Sometimes done at the rules level

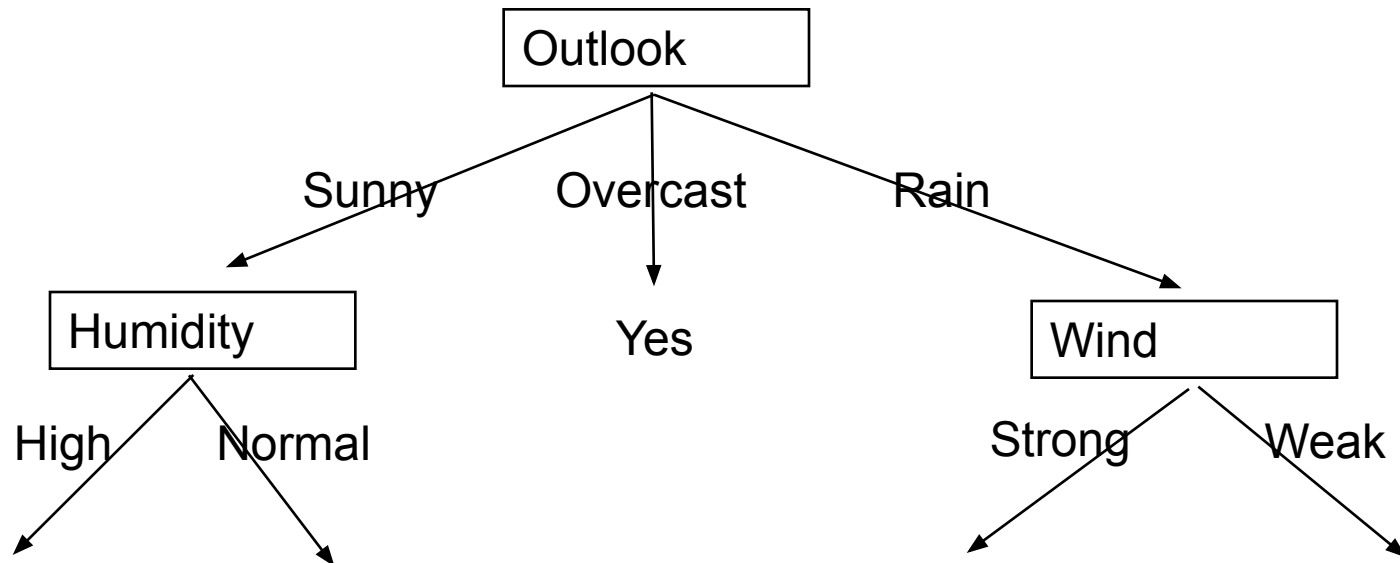
General Strategy: Overfit and Simplify

Rule post-pruning

- Allow tree to grow until best fit (allow overfitting)
- Convert tree to equivalent set of rules
 - One rule per leaf node
 - Prune each rule independently of others
 - ♦ Remove various preconditions to improve performance
 - Sort final rules into desired sequence for use

Example of rule post pruning

- IF (Outlook = Sunny) $\wedge$ (Humidity = High)
 - THEN PlayTennis = No
- IF (Outlook = Sunny) $\wedge$ (Humidity = Normal)
 - THEN PlayTennis = Yes



Extensions of basic algorithm

- Continuous valued attributes
- Attributes with many values
- TE's with missing data
- Attributes with associated costs
- Other impurity measures
- Regression tree

Continuous Valued Attributes

- Create a discrete attribute from continuous variables
 - E.g., define critical Temperature = 82.5
- Candidate thresholds
 - chosen by gain function
 - can have more than one threshold
 - typically where values change quickly

		$(48+60)/2$			$(80+90)/2$	
		↓			↓	
Temp	40	48	60	72	80	90
Tennis?	N	N	Y	Y	Y	N

Attributes with Many Values

- Problem:
 - If attribute has many values, *Gain* will select it (why?)
 - E.g. of birthdates attribute
 - ♦ 365 possible values
 - ♦ Likely to discriminate well on small sample
 - For sample of fixed size n , and attribute with N values, as $N \rightarrow$ infinity
 - ♦ $n_i/N \rightarrow 0$
 - ♦ $-p_i \log p_i \rightarrow 0$ for all i and entropy $\rightarrow 0$
 - ♦ Hence gain approaches max value

Attributes with many values

- Problem: *Gain* will select attribute with many values
- One approach: use *GainRatio* instead

$$\text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitInformation}(S, A)}$$

Entropy of the
partitioning

$$\text{SplitInformation}(S, A) = - \sum_{i=1}^c \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

Penalizes
higher number
of partitions

where S_i is the subset of S for which A has value v_i
(example of $|S_i|/|S| = 1/N$: $\text{SplitInformation} = \log N$)

Unknown Attribute Values

- What if some examples are missing values of attribute A ?
- Use training example anyway, sort through tree
 - if node n tests A , assign most common value of A among other examples sorted to node n
 - assign most common value of A among other examples with same target value
 - assign probability p_i to each possible value v_i of A
 - ♦ assign fraction p_i of example to each descendant in tree
- Classify test instances with missing values in same fashion
- Used in C4.5

Attributes with Costs

- Consider
 - medical diagnosis: BloodTest has cost \$150, Pulse has a cost of \$5.
 - robotics, Width-From-1 ft has cost 23 sec., from 2 ft 10s.
- How to learn a consistent tree with low expected cost?
- Replace gain by
 - Tan and Schlimmer (1990)

$$\frac{Gain^2(S, A)}{Cost(A)}$$

- Nunez (1988)

$$\frac{2^{Gain(S, A)} - 1}{(Cost(A) + 1)^\omega}$$

where $\omega \in [0, 1]$ determines importance of cost

Gini Index

- Another sensible measure of impurity
(i and j are classes)

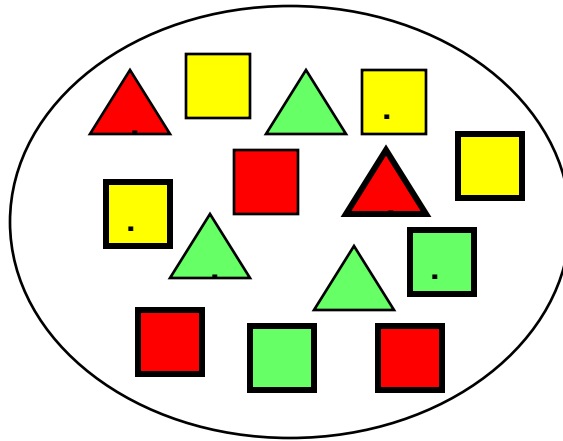
$$Gini = \sum_{i \neq j} p(i)p(j)$$

- After applying attribute A, the resulting Gini index is

$$Gini(A) = \sum_v p(v) \sum_{i \neq j} p(i|v)p(j|v)$$

- Gini can be interpreted as expected error rate

Gini Index



$$p(\square) = \frac{9}{14}$$

$$p(\triangle) = \frac{5}{14}$$

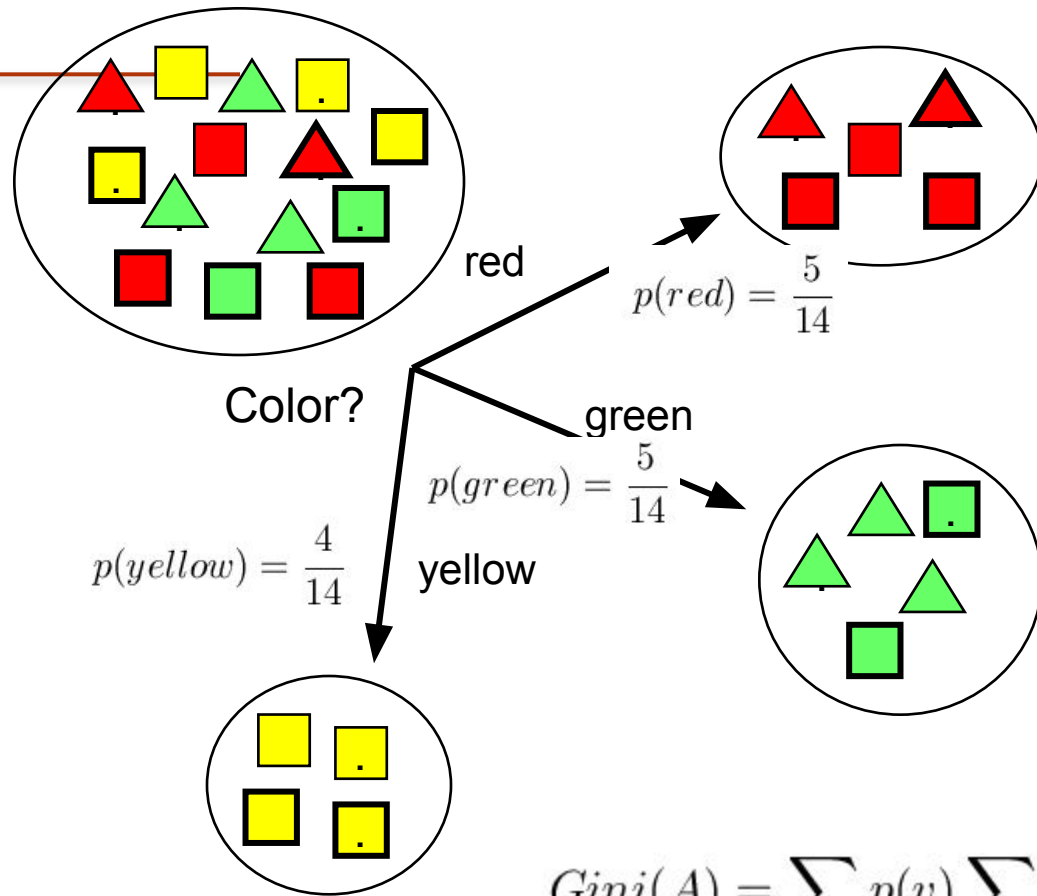
Attributes: color, border, dot

Classification: triangle, square

$$Gini = \sum_{i \neq j} p(i)p(j)$$

$$Gini = \frac{9}{14} \times \frac{5}{14} = 0.230$$

Gini Index for Color



$$Gini(A) = \sum_v p(v) \sum_{i \neq j} p(i|v)p(j|v)$$

$$Gini(\text{Color}) = \frac{5}{14} \times \left(\frac{3}{5} \times \frac{2}{5} \right) + \frac{5}{14} \times \left(\frac{2}{5} \times \frac{3}{5} \right) + \frac{4}{14} \times \left(\frac{4}{4} \times \frac{0}{4} \right) = 0.171$$

Gain of Gini Index

$$Gini = \frac{9}{14} \times \frac{5}{14} = 0.230$$

$$Gini(\text{Color}) = \frac{5}{14} \times \left(\frac{3}{5} \times \frac{2}{5}\right) + \frac{5}{14} \times \left(\frac{2}{5} \times \frac{3}{5}\right) + \frac{4}{14} \times \left(\frac{4}{4} \times \frac{0}{4}\right) = 0.171$$

$$GiniGain(\text{Color}) = 0.230 - 0.171 = 0.058$$

Three Impurity Measures

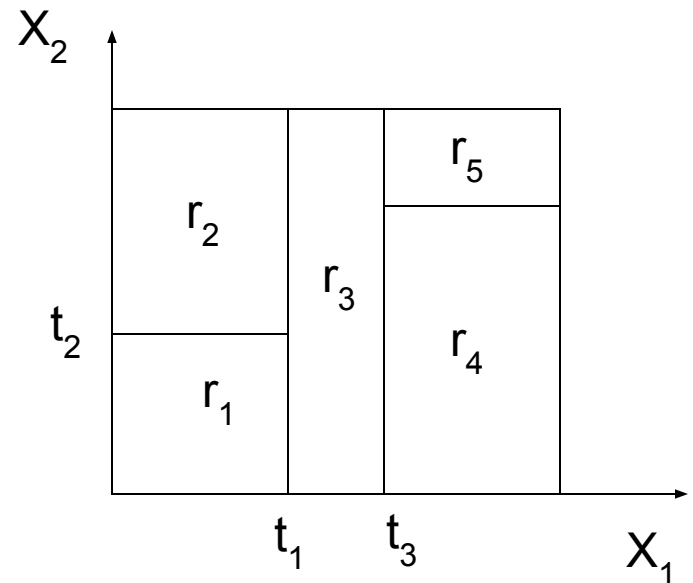
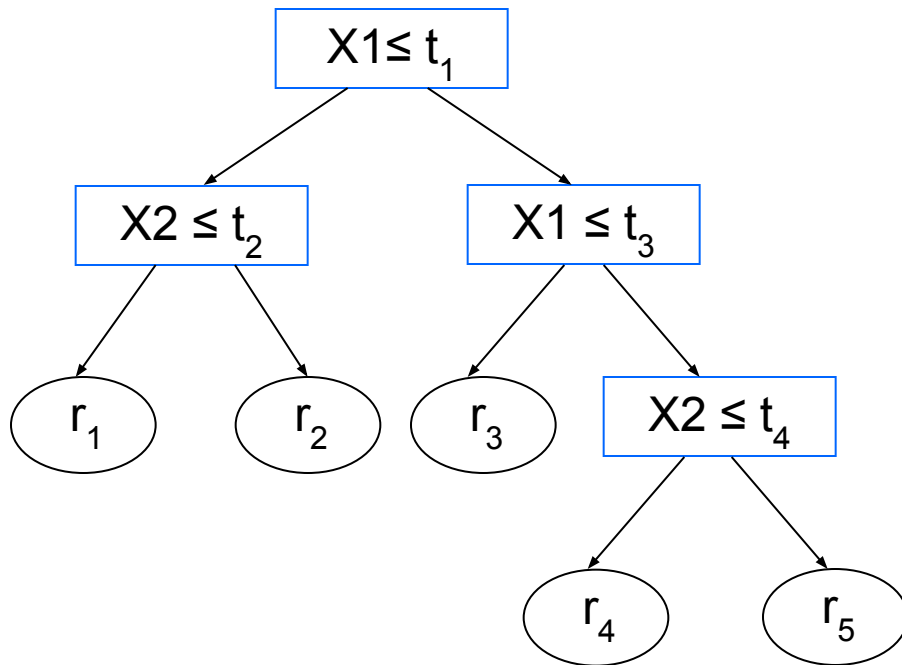
A	Gain(A)	GainRatio(A)	GiniGain(A)
Color	0.247	0.156	0.058
Outline	0.152	0.152	0.046
Dot	0.048	0.049	0.015

Regression Tree

- Similar to classification
- Use a set of attributes to predict the value (instead of a class label)
- Instead of computing information gain, compute the sum of squared errors
- Partition the attribute space into a set of rectangular subspaces, each with its own predictor
 - The simplest predictor is a constant value

Rectilinear Division

- A regression tree is a piecewise constant function of the input attributes



Growing Regression Trees

- To minimize the square error on the learning sample, the prediction at a leaf is the average output of the learning cases reaching that leaf
- Impurity of a sample is defined by the variance of the output in that sample:

$$I(LS) = \text{var}_{y|LS} \{y\} = E_{y|LS} \{(y - E_{y|LS} \{y\})^2\}$$

- The best split is the one that reduces the most variance:

$$\Delta I(L_S, A) = \text{var}_{y|L_S} \{y\} - \sum_a \frac{|L_a|}{|L_S|} \text{var}_{y|L_a} \{y\}$$

Regression Tree Pruning

- Exactly the same algorithms apply: pre-pruning and post-pruning.
- In post-pruning, the tree that minimizes the squared error on VS is selected.
- In practice, pruning is more important in regression because full trees are much more complex (often all objects have a different output values and hence the full tree has as many leaves as there are objects in the learning sample)

When Are Decision Trees Useful ?

- Advantages

- Very fast: can handle very large datasets with many attributes
- Flexible: several attribute types, classification and regression problems, missing values...
- Interpretability: provide rules and attribute importance

- Disadvantages

- Instability of the trees (high variance)
- Not always competitive with other algorithms in terms of accuracy

History of Decision Tree Research

- Hunt and colleagues in Psychology used full search decision trees methods to model human concept learning in the 60's
- Quinlan developed ID3, with the information gain heuristics in the late 70's to learn expert systems from examples
- Breiman, Friedmans and colleagues in statistics developed CART (classification and regression trees simultaneously
- A variety of improvements in the 80's: coping with noise, continuous attributes, missing data, non-axis parallel etc.
- Quinlan's updated algorithm, C4.5 (1993) is commonly used (New:C5)
- Boosting (or Bagging) over DTs is a good general purpose algorithm

Summary

- Decision trees are practical for concept learning
- Basic information measure and gain function for best first search of space of DTs
- ID3 procedure
 - search space is complete
 - Preference for shorter trees
- Overfitting is an important issue with various solutions
- Many variations and extensions possible

Software

- In R:
 - Packages tree and rpart
- C4.5:
 - <http://www.cse.unwe.edu.au/~quinlan>
- Weka
 - <http://www.cs.waikato.ac.nz/ml/weka>